

A Fraction Class with Magic Methods

Let's put some of these magic methods together in a more complex example. [Listing 9-10](#) shows the code of a `Fraction` class. Each `Fraction` object is made up of a numerator (top part) and a denominator (bottom part). The class keeps track of a fraction by storing the separate parts in instance variables, along with the fraction's approximate decimal value. The methods allow the caller to get the reduced value of the fraction, print the fraction along with its floating-point value, compare two fractions for equality, and add two `Fraction` objects.

File: MagicMethods/Fraction.py

```
# Fraction class

import math

class Fraction():
    def __init__(self, numerator, denominator): ❶
        if not isinstance(numerator, int):
            raise TypeError('Numerator', numerator, 'must be an integer')
        if not isinstance(denominator, int):
            raise TypeError('Denominator', denominator, 'must be an integer')
        self.numerator = numerator
        self.denominator = denominator

        # Use the math package to find the greatest common divisor
        greatestCommonDivisor = math.gcd(self.numerator, self.denominator)
        if greatestCommonDivisor > 1:
            self.numerator = self.numerator // greatestCommonDivisor
            self.denominator = self.denominator // greatestCommonDivisor
        self.value = self.numerator / self.denominator

        # Normalize the sign of the numerator and denominator
        self.numerator = int(math.copysign(1.0, self.value)) *
abs(self.numerator)
        self.denominator = abs(self.denominator)

    def getValue(self): ❷
        return self.value

    def __str__(self): ❸
```